



Berkeley UPC - Unified Parallel C

(A joint project of LBNL
and UC Berkeley)



[Home](#) [Downloads](#) [Documentation](#) [Bugs](#) [Publications](#) [Demos](#) [Contact](#) [Internal](#)

Berkeley UPC User's Guide version 2.6.0

☒ Berkeley UPC
Documentation
☐ Berkeley UPC Site-wide

☐ Entire
Web

**Jump to docs for
version:**

[v2.0.1](#) [v2.1.0](#) [v2.2.0](#) [v2.4.0](#) [v2.6.0](#)

Google Search

This guide tells you how to use Berkeley UPC, which is a portable implementation of Unified Parallel C (UPC) that runs on many different parallel systems available today.

Contents

- [UPC standards and APIs supported by this version of Berkeley UPC](#)
- [A Sample UPC Program](#)
- [Compiling UPC programs with 'upcc'](#)
- [Creating libraries of UPC code](#)
- [Running UPC programs with 'upcrun'](#)
- [Using pthreaded UPC Programs](#)
- [Debugging UPC Programs](#)
- [Analyzing UPC Programs with 'upc_trace'](#)
- [Gathering application statistics](#)
- [Profiling UPC Programs with 'upcc -pg' and 'gprof'](#)
- [Berkeley-specific extensions to the UPC Language](#)
 - [Non-blocking and non-contiguous memcpy functions](#)
 - [Value-based collectives convenience interface \(bupc_collectivev.h\)](#)
 - [The 'bupc_all_reduce_all' function family](#)
 - [The 'bupc_dump_shared' function](#)

- [The 'bupc_ptradd' function](#)
 - [The 'bupc_poll' function](#)
 - [The 'bupc_assert_type' built-in](#)
 - [High-precision wall-clock timer support](#)
 - [Runtime thread layout query for hierarchical systems](#)
 - [The 'bupc_atomic*' function family](#)
 - [Platform-specific issues](#)
 - [Known bugs and limitations](#)
-

UPC standards and APIs supported by this version of Berkeley UPC

This version of Berkeley UPC includes

- Full support for [version 1.2 of the UPC language specification](#), (which is itself a superset of ANSI/ISO C99).
 - A complete implementation of the UPC Collectives library, as specified in [version 1.2 of the UPC language specification](#).
 - A reference implementation of the UPC Parallel I/O library, as specified in [version 1.2 of the UPC language specification](#). Note that while this implementation is fully compliant with the specification, it is not designed to be highly performant (all I/O is channeled through a single node). An effort to develop a high performance implementation of the I/O API is underway at [GWU](#).
 - A number of [Berkeley-specific extensions to the UPC Language](#).
-

A Sample UPC Program

Here is a simple hello-world program written in UPC:

```
#include <upc_relaxed.h>
#include <stdio.h>

int main() {
    printf("Hello from thread %i/%i\n", MYTHREAD, THREADS);
    upc_barrier;
    return 0;
}
```

This program prints a message once from each thread (in some arbitrary interleaving), executes a barrier (optional), and exits.

For more involved examples of UPC code, see the [UPC Language Tutorials](#) - from the [UPC Language Community website](#) and the 'upc-examples' directory in of the Berkeley UPC runtime distribution. The [official UPC language specification](#) is a useful reference, and contains a description of the standard libraries.

Compiling UPC programs with 'upcc'

The upcc front end is used to compile UPC programs. It is designed with an interface that is very similar to the standard GNU gcc compiler, for ease of use. For instance, you could compile a physics simulation called 'light' from two source files via

```
upcc -o light particle.upc wave.c -lgrotymath
```

Note that 'wave.c' can contain either UPC code or regular C code, and the 'grotymath' library that is linked into the application can be a regular C library: Berkeley UPC is fully interoperable with regular C source, object, and library files (note: if you compile with the -pthread flag, any C libraries you use must be thread-safe). Berkeley UPC 2.0 also adds support for linking C++/FORTRAN/MPI objects into a UPC executable: see [Mixing C/C++/MPI/FORTRAN with UPC](#).

upcc recognizes most commonly used C compiler flags (-D, -I, etc.). It also uses a number of its own flags for the choice of network API your program will run over, for compiling your UPC code for a static number of threads, and other UPC-specific options. See the [upcc man page](#) for details.

Choosing a network API for your UPC executable

Berkeley UPC executables are always compiled to run over a particular network API. To choose which network API is used, pass the '-network' flag with one of the following values:

Name	Description
lapi	LAPI API for IBM SP networks
gm	GM API for Myrinet networks
elan	elan API for Quadrics networks
vapi	API for Mellanox-based Infiniband networks
sci	SISCI API for Dolphin-based SCI networks (EXPERIMENTAL- currently requires the Linux BigPhysMem kernel patch in order to get more than 1MB of shared heap space)
shmem	SHMEM API for SGI Altix systems and the Cray X1. Other systems providing a SHMEM API may also work, but have not been tested.
portals	Portals API for Cray XT systems running Catamount or CNL.

udp	UDP: works on any system with a standard TCP/IP stack, but is typically slower than using one of the native network types. Generally the fastest option for systems with only Ethernet hardware (notably faster than MPI-over-TCP).
mpi	MPI: works on any system with MPI installed, but is typically slower than using one of the other network types.
smp	"Symmetric multiprocessor (SMP)" mode: uses no network. Currently runs with only a single process, so you must use -pthreads to run with multiple UPC threads.

Note that you can only compile for a given network type if your Berkeley UPC runtime was configured to support it at build/installation time. To see which APIs are supported in your installation, and to see which is used by default, use `'upcc --version'`.

Compiling for a fixed number of UPC threads

The `'-T <number>'` option to `upcc` causes your executable to be build for a fixed number of UPC threads. Alternatively, you can set `'UPCC_FIXED_THREADS=<number>'` in your environment (the `'-T'` flag overrides the environment setting if both are present).

An executable compiled for a fixed number of UPC threads will fail at startup if you try to run it with a different number of threads. However, fixing the number of threads allows optimization on certain operations (such as shared pointer arithmetic), especially when the number of threads is a power of 2.

Overriding global `upcc.conf` settings in your `$HOME/.upccrc` file

`upcc` gets global settings for your installation from a `upcc.conf` file which is created during the configuration stage of a runtime installation. After installation the file is located in the `$prefix/etc` directory of your installation. You can create a `$HOME/.upccrc` file to override any of these settings. See the [upcc man page](#) for a list of available settings.

Berkeley-specific preprocessor macros

Programs compiled with Berkeley UPC will see all of the preprocessor macros provided by your backend C compiler, plus the following:

Name	Value	Description	Standard
<code>__UPC__</code>	1	Defined by any UPC implementation	UPC language
<code>__UPC_VERSION__</code>	Monotonically increasing positive integer constant	UPC specification supported: value is YYYYMM date of that version's ratification (ex: '200310L')	UPC language

<code>__UPC_STATIC_THREADS__</code>	1 if static threads: else undefined	Set to 1 if the '-T' flag was passed to upcc	UPC language
<code>__UPC_DYNAMIC_THREADS__</code>	1 if dynamic threads: else undefined	Set to 1 unless the '-T' flag was passed to upcc	UPC language
<code>__BERKELEY_UPC__</code>	Monotonically increasing positive integer constant	The major version number of the Berkeley UPC release. Example: '1' for release '1.0.3'.	Berkeley UPC only
<code>__BERKELEY_UPC_MINOR__</code>	An integer constant	The minor version number of the Berkeley UPC release. Example: '0' for release '1.0.3'.	Berkeley UPC only
<code>__BERKELEY_UPC_PATCHLEVEL__</code>	An integer constant	The patch version number of the Berkeley UPC release. Example: '3' for release '1.0.3'.	Berkeley UPC only
<code>__BERKELEY_UPC_<NETWORK>_CONDUIT__</code>	1, or undefined	Identifies the network API used. Example: if 'upcc -network=mpi' is used, ' <code>__BERKELEY_UPC_MPI_CONDUIT__</code> ' will be defined, with the value of 1	Berkeley UPC only
<code>__BERKELEY_UPC_PTHREADS__</code>	1, or undefined	Defined to 1 if and only if the '-pthreads' flag is used	Berkeley UPC only
<code>__BERKELEY_UPC_RUNTIME_DEBUG__</code>	1, or undefined	Defined to 1 if and only if a debugging runtime used (i.e. '-g' passed to upcc).	Berkeley UPC only

Using a remote UPC-to-C translator

The upcc front end has the ability to use UPC-to-C translator located on a remote machine. This is provided both as a convenience (the translator takes much longer to build than the runtime, and we provide a public HTTP translator that allows users to get started with Berkeley UPC more quickly), and to support the many systems on which our translator does not build, due to C++ portability issues.

A remote translator can be used either over HTTP, or SSH. To use HTTP, the the 'upcc.cgi' CGI script (located in the 'contrib' directory of the runtime distribution) must be installed and configured with a web server on the remote host. Simply set the 'translator' parameter in your '\$HOME/.upccrc' file (or the global 'upcc.conf') to the URL for the CGI script. To use SSH, you must be able to login to the remote host using SSH, and the 'translator' parameter must be set to 'remote_host:/path/to/translator'. You will want to use key-based authentication, and 'ssh-agent' to avoid entering your password each time you compile. See our [SSH Agent Tutorial](#).

Creating libraries of UPC code

At present you cannot create traditional C-style libraries with UPC code in them using Berkeley UPC (i.e. you cannot successfully use 'ar' to create 'libmyupc.a').

If you wish to create a reusable set of compiled code, you must currently keep the files in *.o format. So, instead of the traditional C format, where you'd create 'libmyupc.a', and then link with something like

```
upcc myprogram.o -L/libpath -lmyupc
```

You must instead do something like

```
upcc myprogram.o /libpath/libmyupc/*.o
```

Running UPC programs

If you compile a UPC program with '`--network=smp`', you can run the executable normally (the same way you'd run 'ls' or 'grep'). Otherwise, you are generating a executable that uses a parallel network API, and this typically means you executable will need some special treatment to be launched correctly.

Berkeley UPC executables should be run the same way as any other parallel program on your system that uses the same underlying network API. So, for instance, a program compiled with '`--network=mpi`' is run on many systems via '`mpirun -np <number of processes> a.out`'. Other systems may use other invocations, such as 'prun' or 'poe', especially when API's other the MPI are used. Consult your system's documentation for details.

Using 'upcrun'

The 'upcrun' script that is installed as part of the Berkeley UPC runtime is our attempt to provide a standard interface for running UPC programs. If your installation has configured 'upcrun.conf' correctly (in many cases the defaults will work), you can run UPC programs portably via commands like

```
upcrun -n 4 parboil
```

This example runs the UPC executable 'parboil' on 4 nodes.

An additional benefit of using upcrun is that it provides consistent support for propagating environment variables to all threads of your UPC program. If you use upcrun, any environment variable beginning with either 'UPC_' or 'GASNET_' is guaranteed to be propagated to all threads. (Support for propagating all environment variables is planned). If you do not use upcrun, environment propagation will only work to the extent that the parallel job launcher you use provides it normally.

You can see how upcrun thinks your job should be run without actually running it by passing the '-t' flag to it. Also, '`upcrun -i <executable>`' will provide information about a Berkeley UPC executable, such as the network API that it was built against, and the number of fixed threads (if any) that it was compiled for.

See '`upcrun --help`' or the [upcrun man page](#) for more information.

Setting the amount of shared memory available to your applications

At startup, each Berkeley UPC thread reserves a fixed portion of its address space (via the `mmap()` system call) for shared memory. This address range can not be used for regular unshared (i.e., `malloc`) memory allocations, and it also serves as a maximum value on the amount of shared memory (per-thread) that the program can use: a UPC program will die with a fatal error if any thread tries to allocate more shared memory than it reserved at startup.

The default amount of shared memory to reserve per UPC thread on a system is chosen at configure time (see the `INSTALL` document in the runtime distribution for details), but you can override that value for a particular application either at compile time, or at startup. Generally this is only needed if you observe that your application is running out of either shared or regular C memory.

To embed a different default amount of shared memory into your application, simply pass `'-shared-heap=144MB'` for instance (to get 144 megabytes per UPC thread). You can also use `'GB'` for gigabyte amounts (if neither `'MB'` nor `'GB'` is used, megabytes are assumed). To override the embedded default amount of shared memory at application startup, set the `UPC_SHARED_HEAP_SIZE` environment variable to whatever value you want (`'2GB'`, etc.), or pass `'-shared-heap'` to `upcrun`.

While it is tempting to simply grab an extremely large shared memory segment, be aware that this is not always a good idea, or even possible. Since the shared address space range cannot be used for regular `malloc` allocations, creating too large of a shared space can cause the amount of regular heap memory available to your application to become small (causing `malloc` to eventually return `NULL` when you request more memory). Also, the shared memory space is reserved via an `mmap()` call, and while this does not generally cause any physical memory pages to be allocated, certain operating systems (for instance, Linux) will not allow more memory to be reserved by applications than the OS can guarantee is available, and so allocating a shared region larger than the physical memory (plus swap space) may fail.

The default amount of shared memory per UPC thread can be changed system-wide by modifying the `'shared_heap'` parameter in the installation's `upcc.conf` file. You can override the system-wide default for your own applications by setting `shared_heap` in your `$HOME/.upccrc` file.

The `upcc.conf` file also provides a `'heap_offset'` parameter (and `upcc` provides a `'-heap-offset'` flag) that affects where the address region for shared memory is located in your program. However, at present it is not useful on any of our supported systems, and so we do not recommend its use.

Using pthreaded Berkeley UPC programs

At present Berkeley UPC programs call network APIs for all inter-process communication, even when processes are located on the same node. While many network APIs perform some kind of optimization for 'local' traffic (avoiding actually putting messages on the network), they are typically slower than simply using shared memory between UPC threads. To provide shared-memory performance within an SMP (or cluster of SMPs), Berkeley UPC supports creating executables that use pthreads to optimize communications between multiple UPC threads running in the same process. To utilize pthreads, pass the `'-pthreads=N'` option to `upcc`, where `N` is the number of processors per node on your system (or configure your `'upcc.conf'` file, as described below). This will cause a single multithreaded process to be run on each node, with shared memory used between UPC threads on the same node. This is the fastest way to run Berkeley UPC programs on SMP systems.

The `-pthread` flag must be passed consistently at all stages of compilation and linking. Also, when pthreads are used, `upcc` needs to delay much of the compilation of your code until link time, so if you split code generation into separate compilation and linking steps (i.e., `'upcc -c foo.upc'`, followed by `'upcc foo.o bar.o'`), you need to pass any macro and/or include path directives (ex: `'-DF00=bar -I/usr/local/include'`) to `upcc` to both the compilation and link commands.

Any C libraries that your code links against must be thread-safe in order to be used with `-pthread`. If one or more of your libraries is not thread-safe, you must compile without pthreads, and run separate processes on the same machine to exploit an SMP system. Currently, such processes will not use any shared memory optimizations, and will communicate with each other via the network API. Support for shared memory between non-pthreaded Berkeley UPC processes will be provided in the near future.

When you link an application with `-pthread`, a subdirectory named `<executable_name>_pthread-Link` will be created in the current directory. This directory exists in order to speed up further linking commands of the same program. If you link the same application again with the same object file names, and none of the global static unshared variables in your program have changed name or size, recompilation of all the files in your application can be avoided, which can make a significant difference in build time for programs with many source files. You may delete the temporary directory at any time without any side effects (other than possibly longer link times).

Unless otherwise specified, pthreaded UPC applications use a default number of pthreads per process (run `'upcc --version'` to see the default for your system. This number is set in the `upcc.conf` configuration file, and can be changed there (or in your `$HOME/.upccrc` file). It can also be overridden in several ways. Compiling with `'upcc -pthread=<NUMBER>'` changes the default number of pthreads per UPC process for an executable to `NUMBER`. If the `'UPC_PTHREADS_PER_PROC'` environment variable is set to a nonzero integer when you run a UPC program, it will override any default value. Finally, `upcrun` is smart about pthreads in several ways. First, if you run a pthreaded parallel job with `'upcrun -n <NUMBER> ...'`, the number of processes actually launched will be divided by the number of pthreads, so that exactly `NUMBER` UPC threads are used. Second, if you use `-network=smp` (which generates a executable that will run only a single process), `upcrun -n NUMBER` will automatically set the number of pthreads to `NUMBER`.

Debugging Berkeley UPC programs

Berkeley UPC programs can now be debugged (with support for UPC-specific constructs) by the "Totalview" debugger produced by [Etnus](#). Totalview version 7.0.1 or greater is required, and support is currently only provided on x86 architectures, using either MPI or Quadrics/elan for the network. See our [tutorial on using Berkeley UPC with Totalview](#) for details.

If you do not have Totalview, you can also use a regular C debugger and get partial debugging support. Berkeley UPC provides several mechanisms for attaching a regular C debugger to one or more of your UPC application's threads at various points during execution. While this does not provide a fully normal debugging environment (the debugger will show the C code emitted by our translator, rather than your UPC code), it can still allow you to see program stack traces and other important information. This can be very useful if you wish to submit a helpful bug report to us. See [Attaching a regular C debugger to Berkeley UPC programs](#) for details.

Berkeley UPC also supports automatically generating backtraces if a fatal error occurs in your program. This will allow you to see a stack trace of the function calls that your program was in at the time it crashed. To use auto-backtracing, run with `upcrun -backtrace` or set `GASNET_BACKTRACE=1` in your environment. The level of backtracing support available depends on the back-end C compiler and operating system, and so not all

systems are equally functional, and some systems will not provide backtraces. See [gasnet/README](https://github.com/berkeleylab/gasnet/blob/master/README) for more information on backtracing.

Analyzing UPC Programs with 'upc_trace'

As of version 2.0, Berkeley UPC includes 'upc_trace', a tool for analyzing the communication behavior of UPC programs. When run on the output of a trace-enabled Berkeley UPC program, 'upc_trace' provides information on which lines of code in your UPC program generated network traffic: how many messages the line caused, what type (local and/or remote gets/puts), what the maximum/minimum/average/combined sizes of the messages were.

Examining tracing information is one of the best ways to go about optimizing your UPC programs. It provides a way for you to see which lines of your code are generating the most network traffic (and the size of the network messages used). From this you may be able to determine how to either avoid some of this traffic, or change your code to use fewer, larger messages (for instance, by replacing sets of individual reads/writes with bulk memory movement calls like 'upc_memget()', etc.), which is typically more efficient. Examining barrier wait times can also let you know if your computations are imbalanced across threads, and/or if you could profit by using split-phase barriers, moving computation in between 'upc_notify' and 'upc_wait'.

How to use 'upc_trace'

1. Tracing must be enabled in order to work. By default, tracing is enabled for debug compilations (i.e. if 'upcc -g' is used), but not otherwise (as it incurs some overhead). If you wish to also trace non-debug executables, you must rebuild your UPC runtime system and pass '- -with-multiconf=+opt_trace' to configure, then build your application with 'upcc -trace'.
2. You must run your application with 'upcrun -trace ...' or 'upcrun -tracefile *TRACE_FILE_NAME* ...'. Either of these flags causes your UPC executable to dump out tracing information while it executes. The '-trace' flag causes one file per UPC thread to be generated, with the name 'upc_trace-a.out.-N', where 'a.out' is the name of your executable, and 'N' is the UPC thread's number. The '-tracefile *NAME*' option lets you specify your own name for the tracing file(s): if the name contains a '%' character, one trace file per thread is generated, with the '%' replaced with the UPC thread's number. Otherwise, all threads will write to the same file.

Note that running with tracing may slow down your application considerably: the exact amount depends on your filesystem, and the ratio of communication/computation in your program. If you are only interested in a subset of trace information, consider setting 'GASNET_TRACEMASK' as described below.

3. After your application has completed, you may run 'upc_trace' on one or more of the trace files generated by your program run:
 1. Running 'upc_trace' on a trace file generated by a single UPC thread shows the information only for that thread. If you pass multiple files from the same application run, the information for the various threads is

coalesced, so passing in all the tracefiles generated by a run allows you to see information for the entire application.

2. There are a number of flags to 'upc_trace' which control what kinds of information is reported, and how it is sorted. See 'upc_trace --help' or the [upc_trace man page](#) for details.
3. Note that upc_trace may take a while to run, especially on large tracefiles. Consider setting GASNET_TRACEMASK and/or GASNET_TRACELOCAL (described below) to streamline the trace file's contents to include only those events you're interested in analyzing.
4. If you compile with 'upcc -opt', it is possible that the UPC-to-C translator has coalesced some of the network operations in your program, in order to get better network performance. This means that 'upc_trace' may not report communication for certain lines of your program, and other lines may seem to be getting/putting more data than they should.

Controlling what gets logged in the trace file by setting GASNET_TRACEMASK

By default, Berkeley UPC will trace all of the following program events:

ID	Feature
G	Network 'gets'. These include both bulk gets (from upc_memget, etc.), and network get operations caused by reading shared memory via shared variables/pointers. The 'g' mask does not include 'local' gets (i.e. writes to shared memory which has affinity to the writing UPC thread), as these do not result in network traffic. Use 'H' to trace local gets.
P	Network 'puts'. These include both bulk puts (from upc_mempu, etc.), and put operations caused by writing to shared memory via variables/pointers. The 'P' mask does not include 'local' puts (i.e. writes to shared memory which has affinity to the writing UPC thread), as these do not result in network traffic. Use 'H' to trace local puts.
B	Barriers, including both blocking (upc_barrier) and non-blocking (upc_notify followed by upc_wait: a pair of these count as a single barrier).
N	Line number information from UPC source files. The "N" and "H" flags must always be among those set for upc_trace to work!
H	Miscellaneous UPC information. The "N" and "H" flags must always be among those set for upc_trace to work! Passing this flag causes the following things to be traced: • UPC lock functions ('upc_lock', 'lock_attempt', and 'upc_unlock'). • UPC collective operations (besides barriers, which are controlled by 'B'). • 'Local' puts/gets, i.e. gets and puts to shared memory which has affinity to the issuing UPC thread (which thus do not result in network traffic). Tracing local gets/puts can significantly expand the size of the trace file (and the time it takes to run 'upc_trace', so if you are not interested in viewing them, consider omitting them from the trace file. You can do this by setting the 'GASNET_TRACELOCAL' environment variable to "no" (or "0"). You may also selectively turn on/off local tracing during program execution by calling the 'bupc_trace_settracelocal()' function (described below). Local get/put tracing only includes accesses performed through shared pointers or the bulk 'upc_memget', etc., functions: it does not include accesses to shared memory made via 'localized' pointers, i.e., shared pointer that have been cast to regular C pointers. • UPC memory allocation operations, i.e., 'upc_alloc', 'upc_all_alloc',

'upc_global_alloc', and 'upc_free' function calls. (Note: allocation operations are not currently reported by 'upc_trace': if you wish to examine where/when your program run has called allocation functions, you must examine the trace file by hand.)

- 'Strict' UPC operations. (Note: 'strict' operations are not currently reported by 'upc_trace': if you wish to examine where/when your program run has executed 'strict' operations, you must examine the trace file by hand.)

To trace only a subset of these features, set the 'GASNET_TRACEMASK' environment variable to a string containing the ID's of the features you wish to trace. **Note that the "N" and "H" flags must always be among those set for 'upc_trace' to work** (if you are intending to manually examine the trace file, they do not need to be set).

So, for instance, if you are trying to perform an analysis that does not require get/put information, you are highly advised to set 'GASNET_TRACEMASK' to "BHN" and 'GASNET_TRACELOCAL' to "no" (or "0"). This will turn off tracing for all get and put operations. Since gets/puts are typically the majority of items in a full trace file, this will probably result in much faster program execution, a much smaller trace file, and faster analysis by 'upc_trace'.

Controlling tracing during runtime

For even more control over tracing, you may call the following functions in your program to set the trace mask dynamically, read its current value, and/or insert your own custom messages into the trace file:

```
extern void      bupc_trace_setmask (const char *newmask);
extern const char * bupc_trace_getmask ();
extern int      bupc_trace_gettracelocal ();
extern void      bupc_trace_settracelocal (int val);
void            bupc_trace_printf ((const char *msg, ...));
```

'bupc_trace_getmask' and 'bupc_trace_setmask' allow programmatic retrieval and modification of the trace masks in effect for the calling thread. The initial values are determined by the 'GASNET_TRACEMASK' environment variables, and the input and output to the mask manipulation functions have the same format as 'GASNET_TRACEMASK' values. **Note that whenever any tracing is enabled (i.e. unless you are temporarily turning off tracing by passing an empty string), the "N" and "H" flags must always be among those set for 'upc_trace' to work.**

'bupc_trace_{get,set}tracelocal' allow the calling thread to programmatically enable/disable tracing of local put/get operations, which correspond to pointer-to-shared accesses that actually have local affinity (and therefore invoke no network communication).

Different UPC threads may set different masks and tracelocal settings, but note that in pthreated UPC jobs all pthreads in a process share these values. These functions have no effect if trace and stats communication profiling are disabled at upcr configure time, or are not enabled for the current run.

Ex:

```
bupc_trace_setmask("PGHN");    // trace everything
bupc_trace_settracelocal(1);  // include local puts and gets
// do something...
bupc_trace_setmask("");        // stop tracing
```

The 'bupc_trace_printf' utility outputs a message into the trace file, if it exists. Note that two sets of parentheses are required when invoking this operation, in order to allow it to compile away completely for

non-tracing builds.

```
Ex:  double A[4] = ...;
      int i = ...;
      bupc_trace_printf(("the value of A[%i] is: %f", i, A[i]));
```

Gathering application statistics

Berkeley UPC also provides the ability to generate a 'stats' report, which contains a statistical summary of program activity. While this report does not give as much information as provided by [tracing](#), it does contain such information as the total number of get/put operations, barriers, etc. (although these cannot be traced back to specific lines of code, as 'upc_trace' provides). But the stats report is generally much smaller than the average trace file, so it may be useful if you are finding that tracing is adding too much overhead to your program runs.

To generate statistics, simply set the 'GASNET_STATSFILE' environment variable to a file name, into which statistics will be written at the end of your program's run. (Note: by default, only debug executables support statistics generation, as it incurs a performance penalty: if you wish to have non-debug UPC executables generate statistics, you must rebuild your UPC runtime system, passing '-with-multiconf=+opt_trace' to configure, then build your application with 'upcc -trace'.) You may generate both stats and tracing info for the same program run if you wish.

Just as with tracing, you may set a mask to control what types of events are included in the statistics, by setting the 'GASNET_STATSMASK' environment variable, and/or by calling the following functions:

```
extern void      bupc_stats_setmask (const char *newmask);
extern const char * bupc_stats_getmask ();
```

The same mask IDs are used by the tracing and statistics masks, i.e., calling 'bupc_stats_setmask ("BP")' would cause execution to gather statistics only for barriers and puts. See the table in the [tracing documentation](#) for the list of IDs.

Profiling UPC Programs with 'upcc -pg' and 'gprof'

The standard GNU 'gprof' profiling tool can be used with Berkeley UPC programs, if your backend C compiler supports gprof (this is autodetected at configure time). Simply compile your UPC program with 'upcc -pg'. When you run the program, one or more 'gmon.out' files are generated (if your UPC program consists of multiple processes, one file per process is created, each in it's own 'gmon.out.process_number' subdirectory). You can then use 'gprof' on one or more of these files (if multiple files are passed, the statistics are combined):

```
upcc -pg foo.c
upcrun -n 2 a.out
gprof a.out gmon.out.0/gmon.out gmon.out.1/gmon.out | less
```

Note that 'gprof' provides timings and statistics for processor usage: it does not include time during which the process has been put to sleep waiting for I/O (including network reads/writes). However, since Berkeley

UPC uses spin-locks in many cases to wait for network events, rather than blocking system calls, you may see that certain 'gasnet *' functions consume large amounts of CPU time. This generally means that your program is spending most of that time waiting for network communication to complete (some fraction is the software overhead inherent in sending/receiving the network traffic). If your program spends a lot of time waiting for network operations to complete, you may be suffering from an imbalanced load across threads (so that some take longer to "catch up" to a barrier, for instance). Restructuring your application may avoid these waiting periods. Or you may be able to use some of this "spare" time for computation (or other network traffic) by switching to use non-blocking barriers (i.e., 'upc_notify/upc_wait'), and/or our [nonblocking memcpy extensions to UPC](#). Replace blocking network constructs (such as 'upc_barrier', 'upc_mempcy', and read/writes to shared variables) with non-blocking equivalents, and insert unrelated computation (and/or network traffic) in between the initialization and completion calls. Of course, you must be able to find unrelated computation/communication for this to work, and the degree to which this is possible will depend on your application.

Berkeley-specific extensions to the UPC Language

Non-blocking and non-contiguous memcpy functions

As of 2.0, Berkeley UPC fully implements a set of non-blocking and non-contiguous extensions to 'upc_mempcy ()' function. These extensions allow you to explicitly overlap memcpy-like functions with computation (and/or with other memcpy calls). They also provide versions that allow you to specify non-contiguous memory regions to get/put.

The full interface is described in our [Proposal for Extending the UPC Memory Copy Library Functions](#). See that document for details on the functions and their usage.

Value-based collectives convenience interface (bupc_collectivev.h)

This library wrapper provides a value-based convenience interface to the UPC collectives library that is part of UPC 1.2. There is a small amount of optimization for Berkeley UPC, but the wrapper is generic and can be used with any fully UPC-1.2 compliant implementation of the UPC collectives library. All operations are implemented as thin wrappers around that library. In most cases, operands to this library are simple values, and nothing is required to be single-valued except for the data type in use and the root thread identifier (in the case of rooted collectives). The purpose of this wrapper is to provide convenience for scalar-based collective operations, especially in cases where there are not multiple values available to be communicated in aggregate (in which case the full array-based UPC collectives interface is likely to use fewer messages and achieve better performance) or for use in setup code (where performance is secondary to simplicity). See the [collectivev documentation](#) for full interface details.

The 'bupc_all_reduce_all' function family

This is an extension to the [UPC Collectives Specification](#). The 'bupc_all_reduce_all' functions behave identically to the 'upc_all_reduce' functions, except that the 'dest' argument has the semantics of the 'dest' argument to 'upc_all_broadcast', i.e. the result of the reduction is broadcast to all thread, instead of just one.

The 'bupc_dump_shared' function

Shared pointers in UPC are logically composed of three fields: the address of the data that the shared pointer currently points to, the UPC thread on which that address is valid, and the 'phase' of the shared

pointer (see the [official UPC language specification](#) for an explanation of shared pointer phase). Our version of UPC provides a 'bupc_dump_shared' function that will write a description of these fields into a character buffer that the user provides:

```
int bupc_dump_shared(shared const void *ptr, char *buf, int maxlen);
```

Any pointer to a shared type may be passed to this function. The 'maxlen' parameter gives the length of the buffer pointed to by 'buf', and this length must be at least BUPC_DUMP_MIN_LENGTH, or else -1 is returned, and errno set to EINVAL. On success, the function returns 0, The buffer will contain either "<NULL>" if the pointer to shared == NULL, or a string of the form

```
"<address=0x1234 (addrfield=0x1234), thread=4, phase=1>"
```

The 'address' field provides the virtual address for the pointer, while the 'addrfield' contains the actual contents of the shared pointer's address bits. On some configurations these values may be the same (if the full address of the pointer can be fit into the address bits), while on others they may be quite different (if the address bits store an offset from a base initial address that may differ from thread to thread).

Both bupc_dump_shared() and BUPC_DUMP_MIN_LENGTH are visible when any of the standard UPC headers (upc.h, upc_relaxed.h, or upc_strict.h) are #included.

The 'bupc_ptradd' function

Blocked pointers-to-shared in UPC are currently restricted to being declared with a compile-time constant block size. This can present problems in situations where the block size of a given array is input-dependent or otherwise unknown at compile time, and one wishes to conveniently access the array elements in layout order according to a specific block size.

The 'bupc_ptradd()' function provides support for performing pointer-to-shared arithmetic with general blocksize, which need not be compile-time constant.

```
shared void * bupc_ptradd(shared void *p, size_t blockelems, size_t
elemsz, ptrdiff_t elemincr);
- 'p': the base pointer
- 'blockelems': the block size (number of elements in a block)
- 'elemsz': the element size (usually sizeof(*p))
- 'elemincr': the positive or negative offset from the base pointer
```

The following call:

```
upc_ptradd(p, blockelems, sizeof(T), elemincr);
```

Returns a value q as if it had been computed:

```
shared [blockelems] T *q = p;
q += elemincr;
```

however, the blockelems argument is not required to be a compile-time constant. Blockelems must be non-negative, but may be zero to indicate an indefinite blocking factor. Here's an example of indexing into a dynamically-allocated array whose block size is not known until run time.

```
int blockelems = ...; // choose some arbitrary block size
```

```
// allocate an array of doubles with that blocksize
shared void *myarr = upc_all_alloc(..., blockelems*sizeof(double));

// access element 14
double d = *(shared double *)bupc_ptradd(myarr, blockelems, sizeof(double), 14);
```

It's worth noting that in some cases `bupc_ptradd()` may be less efficient than regular pointer-to-shared addition, because the compile-time constant blocksize of the pointer referent type generally makes the latter more amenable to compiler optimization of the addition operation and surrounding code. This is especially true in the case of indefinitely blocked or cyclically blocked pointers-to-shared. However, the cost may be worth the added convenience in non-performance-critical code.

The 'bupc_poll' function

The `'bupc_poll()'` function explicitly causes the UPC runtime to attempt to make progress on any network requests that may be pending.

You will normally not need to call this function, as the runtime will automatically perform checks for incoming network requests whenever your UPC code causes network activity to be performed, and this usually occurs fairly frequently in a UPC application. However, if you writing your own 'spin lock' style synchronization, you may need to use this function to avoid deadlock. Here is an example:

```
shared strict int flag[THREADS];

...

if (MYTHREAD % 2) {
    while (flag[MYTHREAD] == 0)
        bupc_poll();
} else {
    ... some calculation ...
    flag[MYTHREAD - 1] = 1;
}
```

Here the 'even' UPC threads are performing some calculation, then informing the 'odd' threads that the result is ready by setting a per-thread flag. If the `'bupc_poll()'` were omitted, the 'odd' threads might (on certain platforms/networks) consume all of the CPU forever in the 'while' test, never checking for the incoming network message that would set `flag[MYTHREAD]`.

If a program contains computationally intensive sections in which no remote accesses are performed for a long time, it is also possible that performance may be improved by intermittently calling `bupc_poll`, particularly if other threads are likely to be performing remote accesses (or memory allocation requests) during this time.

The 'bupc_assert_type' built-in

The `'bupc_assert_type(expr, type)'` built-in operation allows testing for compile-time type equality, and is primarily used by our UPC compiler test suite.

1. 'expr' = any arbitrary (legal) UPC expression
2. 'type' = any legal C/UPC type

If 'expr' has a static type which is identical to 'type', does nothing. Otherwise, prints a non-fatal warning containing the line number and a description of the two differing types.

High-precision wall-clock timer support

```
typedef      bupc_tick_t; /* 64-bit integral type */
#define      BUPC_TICK_MAX
#define      BUPC_TICK_MIN
bupc_tick_t  bupc_ticks_now ();
uint64_t     bupc_ticks_to_us (bupc_tick_t ticks);
uint64_t     bupc_ticks_to_ns (bupc_tick_t ticks);
double       bupc_ticks_granularityus ();
double       bupc_ticks_overheadus ();
```

The 'bupc_tick_t' type and associated functions provide portable support for querying high-precision system timers for obtaining wall-clock timings of sections of code. Most CPU hardware offers access to high-performance timers with a handful of instructions, providing timer precision and overhead that can be several orders of magnitude better than can be obtained through the use of the `gettimeofday()` system call.

The 'bupc_tick_t' type represents an integral quantity of abstract timer ticks, whose ratio to real time is system-dependent and thread-dependent. `bupc_ticks_now()` returns the current value of the tick timer for the calling thread, using the fastest mechanism available. `bupc_ticks_to_us()` and `bupc_ticks_to_ns()` convert a difference in `bupc_tick_t` values obtained by the calling thread into microseconds or nanoseconds, respectively. The `bupc_ticks_to_{us,ns}()` conversion calls can be significantly more expensive than the `bupc_ticks_now()` tick query, so for timing short intervals it's recommended to keep timing results in units of ticks until final output. `BUPC_TICK_MAX` and `BUPC_TICK_MIN` provide tick values which are respectively larger and smaller than any possible tick value. `bupc_ticks_granularityus()` and `bupc_ticks_overheadus()` respectively report the estimated microsecond granularity (minimum time between distinct ticks) and microsecond overhead (time it takes to read a single tick value, not including conversion) for the timer facility.

Example:

```
bupc_tick_t start = bupc_ticks_now();
    compute_foo(); /* do something that needs to be timed */
bupc_tick_t end = bupc_ticks_now();

printf("Time was: %d microseconds\n", (int)bupc_ticks_to_us(end-start));

printf("Timer granularity: <= %.3f us, overhead: ~ %.3f us\n",
    bupc_ticks_granularityus(), bupc_ticks_overheadus());
printf("Estimated error: +- %.3f %%\n",
    100.0*(bupc_ticks_granularityus()+bupc_ticks_overheadus()) /
    bupc_ticks_to_us(end-start));
```

It's important to keep in mind that raw `bupc_tick_t` values are thread-specific quantities with a thread-specific interpretation (e.g. they might represent a hardware cycle count on a particular CPU, starting at some arbitrary time in the past). More specifically, raw ticks do **NOT** provide a globally-synchronized timer (i.e. the simultaneous absolute tick values may differ across threads), and furthermore the tick-to-wallclock conversion ratio might also differ across threads (e.g. on a cluster with heterogeneous CPU clock rates, the raw tick values may advance at different rates for different threads). Therefore as a rule of thumb, raw `bupc_tick_t` values and `bupc_tick_t` intervals obtained by **different** threads should never be directly compared or arithmetically combined, without first converting the relevant tick intervals to wall time intervals.

Runtime thread layout query for hierarchical systems


```

unsigned int bupc_thread_distance(int threadX, int threadY);
#define      BUPC_THREADS_SAME
#define      BUPC_THREADS_VERYNEAR
#define      BUPC_THREADS_NEAR
#define      BUPC_THREADS_FAR
#define      BUPC_THREADS_VERYFAR

```

bupc_thread_distance takes two thread identifiers (whose values must be in 0..THREADS-1, otherwise behavior is undefined), and returns an unsigned integral value which represents an approximation of the abstract 'distance' between the hardware entity which hosts the first thread, and the hardware entity which hosts the memory with affinity to the second thread. In this context 'distance' is intended to provide an approximate and relative measure of expected best-case access time between the two entities in question. Several abstract 'levels' of distance are provided as pre-defined constants for user convenience, which represent monotonically non-decreasing 'distance':

- BUPC_THREADS_SAME (must be defined to 0) implies threadX == threadY
- BUPC_THREADS_VERYNEAR implies threadX has the closest possible distance (fastest access) to threadY's memory without being the same actual thread
- BUPC_THREADS_NEAR implies distance not less than BUPC_THREADS_VERYNEAR, but not more than BUPC_THREADS_FAR
- BUPC_THREADS_FAR implies distance not less than BUPC_THREADS_NEAR, but not more than BUPC_THREADS_VERYFAR
- BUPC_THREADS_VERYFAR implies threadX has the farthest possible distance (slowest access) to threadY's memory

These constants have implementation-defined integral values which are monotonically increasing in the order given above. Implementations may add further intermediate level with values between BUPC_THREADS_VERYNEAR and BUPC_THREADS_VERYFAR (with no corresponding define) to represent deeper hierarchies, so users should test against the constants using <= or >= instead of ==.

The intent of the interface is for users to not rely on the physical significance of any particular level and simply test the differences to discover which threads are **relatively** closer than others. Implementations are encouraged to document the physical significance of the various levels whenever possible, however any code based on assuming exactly N levels of hierarchy or a fixed significance for a particular level will probably not be performance portable to different implementations or machines.

The relation is symmetric, ie:

```
bupc_thread_distance(X,Y) == bupc_thread_distance(Y,X)
```

but the relation is **not** transitive: bupc_thread_distance(X,Y) == A && bupc_thread_distance(Y,Z) == A does **NOT** imply bupc_thread_distance(X,Z) == A

Furthermore, the value of bupc_thread_distance(X,Y) is guaranteed to be unchanged over the span of a single program execution, and the same value is returned regardless of the thread invoking the query.

The 'bupc_atomic*' function family

```

type bupc_atomicX_read_relaxed(shared void *ptr);
type bupc_atomicX_read_strict(shared void *ptr);
void bupc_atomicX_set_relaxed(shared void *ptr, type val);

```

```
void bupc_atomicX_set_strict(shared void *ptr, type val);
type bupc_atomicX_fetchadd_relaxed(shared void *ptr, type op);
type bupc_atomicX_fetchadd_strict(shared void *ptr, type op);
```

Where *type* and *X* take on the values of each pair from the following table:

Type	X
uint64_t	U64
int64_t	I64

This family of functions provide atomic read, write and read-modify-write of the indicated data types. When these functions are used to access a memory location in a given synchronization phase, atomicity is guaranteed if and only if no other mechanisms are used to access the same memory location in the same synchronization phase. Memory accesses are relaxed or strict as indicated by the function names. The `fetchadd` functions atomically add the second argument to the location given by the first argument and return the value prior to the addition.

Support for additional data types (both integer and floating point) and fetch-and-OP operations are expected to appear in a future release.

Known bugs and limitations

This release of Berkeley UPC has a number of known limitations and bugs:

Preprocessor macros defined in UPC files must not affect .h files

Berkeley UPC translates your UPC programs into C code, then runs a regular C compiler on your system to generate object code. To avoid handling vendor-specific inline assembly code that appears in some header files on many of the various systems we run on, we currently have our UPC-to-C translator 'put back' all non-UPC header files (i.e., .h files which don't contain any UPC constructs), which are then handled by the regular C compiler (we do not support placing inline assembly in your UPC code). A side effect of this process is that the preprocessor is run twice on your program. Since any `#defined` macros you place in your UPC code are expanded (and their definitions forgotten) the first time the preprocessor is run, these macros will not be present the second time .h files are included. Thus, UPC code such as

```
#define NDEBUG
#include <assert.h>
```

will not work as expected if the `NDEBUG` definition modifies the behavior of `assert.h` (which, in this example, it does: this `NDEBUG/assert.h` case is the most common case where users run into this issue with our compiler).

There is a simple workaround: if you need to define a macro that affects the behavior of `#included` files, define it on the command line to `upcc`:

```
upcc -DNDEBUG myprogram.upc
```

Behavior of the 'getenv()/setenv()' functions

It is not well-defined in the UPC specification whether the standard `getenv()` function should return the same values on all threads, and/or if these values should include those present in the environment of the process that launches the UPC application.

Berkeley UPC guarantees that `getenv()` allows retrieval of certain environment variable values that were present when the job was launched. At present this function is only guaranteed to retrieve these value for all threads if the environment variable's name begins with `'UPC_'` or `'GASNET_'`. On some platforms all environment variables seen by the job launcher may be propagated, but it is not portable to rely on this.

The `'setenv()'` and `'unsetenv'` functions are not guaranteed to work in a Berkeley UPC runtime environment, and should be avoided.

Correctness when using GCC 4.x as the C compiler

There is a known correctness problem in the gcc 4.x optimizer that may affect correctness of shared-local accesses in UPC (i.e., shared accesses that result in node-local accesses at runtime). In a nutshell, it's possible that in rare cases gcc 4.x may misoptimize a shared-local access such that it deterministically reads or writes an incorrect value. If you suspect you may be encountering this issue, the following actions are recommended for diagnosis:

1. Try compiling your code in debug mode (ie with `'upcc -g'`). If the problem persists, then this issue is **not** the culprit.
2. Try compiling your code using the flag `'upcc -Wc,-fno-strict-aliasing'`. If the problem persists, then this issue is **not** the culprit.
3. Run your code several times. If the problem is intermittent, then this issue is probably not the culprit (the optimizer bug is deterministic).

If you still believe you are encountering this issue, there are several recommended workarounds:

1. Configure BUPC to use a different backend C compiler. If you have a non-gcc vendor C compiler available, this may actually be a better choice for performance anyhow. Failing that, using gcc 3.x should also resolve the issue, as the bug is only believed to be present in gcc 4.x.
2. Build the affected modules using the flag `'upcc -Wc,-fno-strict-aliasing'`. This makes the gcc 4.x optimizer more conservative, and also inhibits the illegal optimization.
3. Reconfigure BUPC using `'configure --enable-conservative-local-copy'`. This globally activates a more conservative implementation of shared-local accesses that also prevents the illegal optimization.

The performance impact of the workarounds above is expected to be application-dependent.

The next version of BUPC will include a restructuring of the way shared-local accesses are performed at the C code level. This restructuring is motivated by performance concerns, but we expect that as a side-effect it will also workaround this gcc optimizer bug.

GCCUPC+UPCR with -pthreads

GCCUPC+UPCR has a known problem in `-pthreads` compilation mode, whereby programs with a significant amount of statically-allocated private data may fail at program initiation time with an error message like:

```
UPC Runtime error: pthread_create: Invalid argument
```

Users encountering this error are recommended to workaround it by either using the BUPC translator (which does not demonstrate the problem), or reworking their program to use less statically-allocated private data.

Other known limitations/bugs

- The UPC-to-C translator currently does not perform any optimizations on the network traffic of UPC programs--

all network accesses are treated as 'strict', and are blocking. *Note: we now have some experimental optimizations implemented, but they are not considered production-ready, and so are not enabled by default. Pass the ' -opt ' flag to upcc to turn on these translator optimizations.*

- Shared memory optimizations for inter-thread communications are used on SMP systems only if you use the '-pthreads' flag to upcc. This is not possible for applications which need to use non-thread-safe libraries, and so such programs will instead use the network API to communicate between UPC threads on the same machine. Support for System V shared memory optimizations between unthreaded UPC processes is planned.
- Programs which #include complex.h, and/or tgmath.h do not work on certain platforms.
- Inline assembly statements are not supported within UPC code. However, UPC applications are permitted to #include C-mode header files containing inline assembly statements, written in the syntax appropriate for the backend native C compiler and platform. (A C-mode header file is any header which contains no UPC-specific constructs and includes no files containing UPC-specific constructs.)
- UPC programs are not permitted to register signal handlers (i.e. via signal(2) or sigaction(2)), use the pthread library (ie. pthread_create(3), pthread mutexes), fork child processes (via fork(2)/clone(2)), call setjmp/longjmp, or use System V shared memory constructs.
- UPC programs are recommended to terminate by calling exit(), abort(), upc_global_exit() or returning from main. Calling _exit() could result in the creation of orphaned processes.
- UNIX file descriptors (obtained via open(), fopen(), socket() etc) may not be safely shared across UPC threads.
- 'stdin' may not work on some platforms - programs are recommended to obtain their input via the command line or file system. 'stdout' and 'stderr' are multiplexed on a line-by-line basis to the console, but not ordered in any way between UPC threads (i.e., a single thread's output lines will appear in order, but may be arbitrarily interleaved with the output from other threads)"
- The variable-argument list access macro va_arg may be used with pointer-to-shared types, but the pointer type passed to va_arg must match the type of the actual argument (specifically, the shared blocksize must match) and no typedefs may be used in the argument to va_arg.
- Berkeley UPC does not currently support C99-style variable-length arrays (eg a stack temporary array with bounds that are not a compile-time constant). This is planned to be fixed in a future version.
- Many other bugs with particular code constructs, and/or when using specific architectures/compilers/networks. See our [Bugzilla bug database](http://bugzilla.lbl.gov) for the grisly details.

Platform-specific issues

Running into Maximum size limits on pinning-based networks

On systems that pin RDMA-addressable memory (such as Myrinet and Infiniband), the amount of shared memory that a default Berkeley UPC build can provide to a UPC program will be no larger than the maximum region that the OS and network drivers allow to be pinned at once. While this is typically a large fraction of physical memory, it may prove insufficient for your application. In this case, a "large segment" mode is available, which is slightly less fast in some situations, but which provides the maximum possible UPC shared memory space. To use large segment mode, the Berkeley UPC runtime needs to be reconfigured with '-enable-segment-large', and rebuilt.

RDMA-enabled lapi-conduit with non-uniform pthread layouts

A known bug in the new lapi-conduit RDMA support will cause crashes at startup for jobs where the number of pthreads per node differs across nodes (ie for 'upcrun -np T -p P' where $T \bmod P \neq 0$, or where the user has manually provided a UPC_PTHREADS_MAP environment setting specifying a non-uniform layout).

The suggested workaround is either to select a uniform pthread layout, or to set GASNET_LAPI_USE_RDMA=0 in the environment to disable the RDMA support.

IBM SP segment size limits (in 32 bit mode)

32 bit applications run on the IBM SP by default are given a small (256MB) memory segment which contains the program stack, heap, and static data. If your stack/heap/data exceed this size, your program will fail with a message stating "there is not enough memory available". To run such programs you must enable either "large program support" (supports up to 2 GB of program data: available on AIX 4.3 and later) or "very large program support" (supports up to 3.25 GB of program data: available on AIX 5.1 and later). Enabling these modes requires passing the '-bmaxdata' data flag to the IBM linker, so use upcc's '-wl,' flag:

```
upcc -v -wl,-bmaxdata:0x80000000 foo.c      # 'large'
```

```
upcc -v -wl,-bmaxdata:0x80000000/dsa foo.c  # 'very large'
```

Users wishing to know more details about this issue are invited to [search for AIX "large program support" on Google](#).

In general, for performance reasons (such as a faster shared pointer implementation), Berkeley UPC users are encouraged to build 64-bit UPC programs on IBM SP systems, in which case these memory segment limits are not an issue. Instructions on how to build a 64-bit Berkeley UPC compiler on the IBM SP are available in the INSTALL document that comes with the distribution. To see if the copy of upcc you're running is 32/64 bit, check the 'Binary interface' field in the output of 'upcc -version'.

Feedback

Please [contact us](#) with your bug reports, comments, and suggestions.

Thank you for using Berkeley UPC!

[Home](#) [Downloads](#) [Documentation](#) [Bugs](#) [Publications](#) [Demos](#) [Contact](#) [Internal](#)
